

ENTITY RELATIONSHIP DIAGRAM

CarSage

Database Schema — Supabase (PostgreSQL)

Prepared by: Mohamad Dib | Version: 1.0 | Date: September 2026

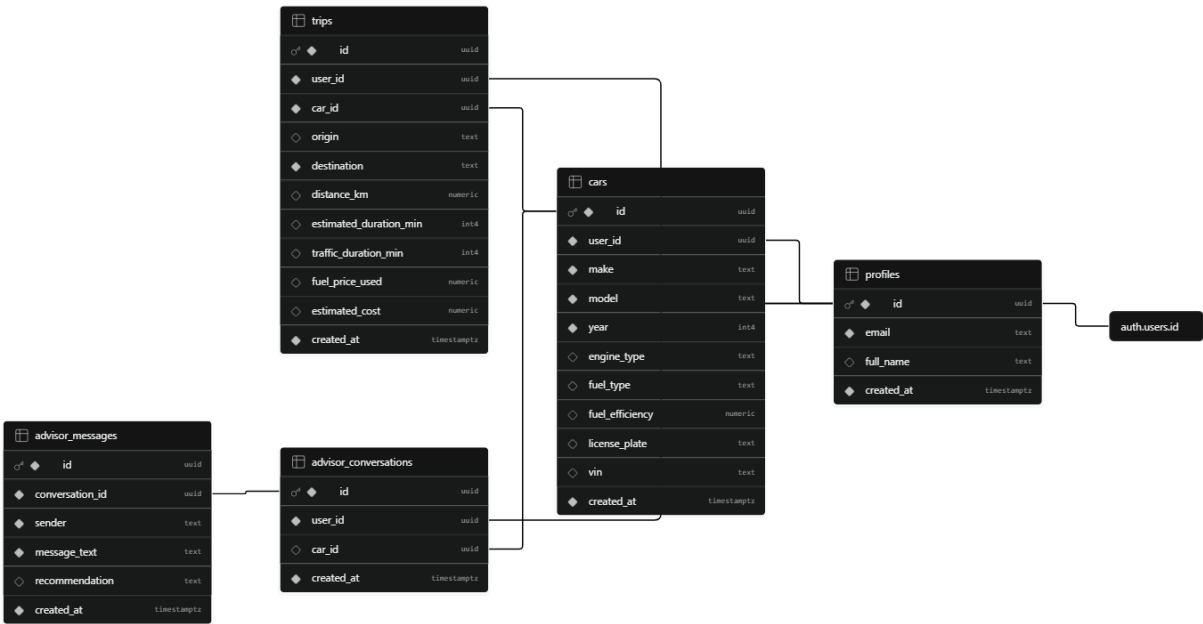
1. Overview

This document defines the entity relationship structure for the CarSage database, implemented in Supabase (PostgreSQL). It covers all tables, primary and foreign keys, relationships, and field-level definitions needed to support the two MVP core features (Trip Planner and AI Advisor) and their supporting basics (Car Onboarding and Car Profile).

The schema below has already been implemented and deployed in the live CarSage Supabase project, with row-level security (RLS) enabled on every table so that each user can only access their own data.

2. Diagram

The diagram below is a live export from the Supabase Studio schema visualizer, generated directly from the deployed database.



3. Entity Definitions

3.1 profiles

Extends Supabase's managed auth.users table with app-specific profile data. One row is created automatically for every registered user via a database trigger.

Field	Type	Key	Description
id	uuid	PK / FK	References auth.users(id). Primary key of this table.
email	text	—	User's email address, copied from auth.users on signup.
full_name	text	—	User's display name (optional).
created_at	timestampz	—	Timestamp the profile was created.

3.2 cars

Stores each user's vehicle(s), populated during Car Onboarding and viewed/edited on the Car Profile screen.

Field	Type	Key	Description
id	uuid	PK	Primary key, auto-generated.
user_id	uuid	FK	References profiles(id). The owner of this car.
make	text	—	Vehicle manufacturer (e.g., Toyota).
model	text	—	Vehicle model (e.g., Corolla).
year	int4	—	Model year.
engine_type	text	—	e.g., petrol, diesel, hybrid, electric.
fuel_type	text	—	Fuel grade/type used by the vehicle.
fuel_efficiency	numeric	—	Known fuel efficiency, used in Trip Planner cost calculations.
license_plate	text	—	Vehicle registration plate.
vin	text	—	Vehicle Identification Number.
created_at	timestampz	—	Timestamp the car was added.

3.3 trips

Stores each Trip Planner query and its calculated results, linked to both the user and the specific car used.

Field	Type	Key	Description
id	uuid	PK	Primary key, auto-generated.
user_id	uuid	FK	References profiles(id).
car_id	uuid	FK	References cars(id). The car used for this trip.
origin	text	—	Starting point (optional).
destination	text	—	Trip destination.
distance_km	numeric	—	Route distance from the Maps API.
estimated_duration_min	int4	—	Baseline travel time estimate.
traffic_duration_min	int4	—	Traffic-adjusted travel time estimate.
fuel_price_used	numeric	—	Local fuel price used in the cost calculation.

Field	Type	Key	Description
estimated_cost	numeric	—	Calculated estimated fuel cost for the trip.
created_at	timestamptz	—	Timestamp the trip estimate was generated.

3.4 advisor_conversations

Represents one AI Advisor session. A user may have multiple conversations over time, optionally tied to a specific car for context.

Field	Type	Key	Description
id	uuid	PK	Primary key, auto-generated.
user_id	uuid	FK	References profiles(id).
car_id	uuid	FK (nullable)	References cars(id). Optional car context for the conversation.
created_at	timestamptz	—	Timestamp the conversation started.

3.5 advisor_messages

Stores each individual message within an AI Advisor conversation, including the AI's DIY-vs-mechanic recommendation where applicable.

Field	Type	Key	Description
id	uuid	PK	Primary key, auto-generated.
conversation_id	uuid	FK	References advisor_conversations(id).
sender	text	—	Either 'user' or 'ai'.
message_text	text	—	The message content.
recommendation	text (nullable)	—	'diy', 'mechanic', or null if not applicable to this message.
created_at	timestamptz	—	Timestamp the message was sent.

4. Relationships Summary

- profiles (1) — (many) cars: a user can own multiple cars.
- profiles (1) — (many) trips: a user can plan multiple trips.
- cars (1) — (many) trips: a car can be used in multiple trips.
- profiles (1) — (many) advisor_conversations: a user can start multiple AI Advisor conversations.
- cars (1) — (many) advisor_conversations: a car can be the subject of multiple conversations.
- advisor_conversations (1) — (many) advisor_messages: a conversation contains multiple messages.
- auth.users (1) — (1) profiles: every Supabase-managed auth user has exactly one profile row.

5. Security Notes

- Row-Level Security (RLS) is enabled on all five tables.

- Policies restrict select/insert/update/delete so a user can only access rows where user_id (or, for advisor_messages, the parent conversation's user_id) matches their own authenticated ID.
- The handle_new_user() trigger function that auto-creates profile rows is hardened with a fixed search_path and no public execute grant, per Supabase security advisor recommendations.